Experimental Design for Carbon-Aware Orchestrator Publication

1 Comparative Evaluation Framework

Algorithms to Compare:

1. Carbon-Aware Orchestrator
2. Kubernetes Default Scheduler
3. Random Placement
4. First-Fit/Bin-Packing Approach
   - Process workloads in a predefined order (typically arrival time). For each workload, scan through nodes sequentially. Place the workload on the first node with sufficient resources. If no node has capacity, reject the workload or wait.
   - Differences from CAO:
     - Only considers where to place workloads, not when
     - Optimizes for bin utilization, not carbon emissions
     - Makes greedy, immediate decisions without considering future timeslots
     - Has no concept of deadlines or scheduling windows
5. Theoretical optimal solution (if computationally feasible)
6. Other state-of-art energy-aware schedulers

2 Workload Diversity

Synthetic Workloads

Parameterized by characteristics:

– CPU/Memory intensity ratios

– Execution duration (short vs. long-running)

– Deadlines (strict vs. flexible)

– Burstiness patterns

Real-World Traces (tbd)

- Alibaba or Google cluster traces
- CI/CD pipeline workloads
- Batch processing workloads
- ML training jobs

3 Key Metrics

Primary Carbon Metrics

- Total carbon emissions (kg CO_2e)
- Carbon reduction percentage vs. baselines
- Carbon efficiency (computation/kg CO_2e)

System Performance

- Resource utilization across nodes/regions
- Job completion times
- SLA violations/deadline misses
- Scheduling overhead (time complexity)

Tradeoff Analysis

- Carbon savings vs. performance impact
- Pareto efficiency plots

4 Experimental Variables

Systematically vary:

- **Infrastructure Scale:** 4, 8, 16, 32, 64 nodes
- **Regional Distribution:** Single-region vs. multi-region
- **Carbon Intensity Patterns:** Static, time-varying, region-varying
- **Scheduling Window Flexibility:** 0h, 24h, 48h, 72h

5 Statistical Validity

– Run each experiment configuration 30+ times

– Report mean, median, standard deviation, and 95% confidence intervals

– Conduct statistical significance tests (e.g., t-tests, ANOVA)

– Perform sensitivity analysis for key algorithm parameters

6 Practical Impact

– Demonstrate real-world applicability through case studies

– Show financial cost implications (carbon pricing)

– Calculate environmental impact at scale (projections)

7 Experimental Results and Analysis

We conducted extensive experiments to evaluate the performance characteristics and carbon efficiency of our carbon-aware orchestration algorithm. The following sections present our key findings across multiple dimensions.

7.1 Algorithm Performance Characteristics

Figure 1 illustrates how our algorithm's execution time scales with the number of pods processed. The majority of execution times remain under 20ms regardless of workload size, demonstrating good algorithmic efficiency for typical scenarios. However, we observe an outlier at call 11 (processing 7 pods) that exhibits significantly higher execution time (approximately 17 seconds). This outlier could be attributed to several factors:

- Complex interactions between specific pod characteristics and scheduling constraints
- Resource contention on the host machine during that particular test run
- Search space explosion for specific combinations of pods and constraints

To better understand the algorithm's per-pod efficiency, Figure 2 shows the execution time normalized by the number of pods processed.

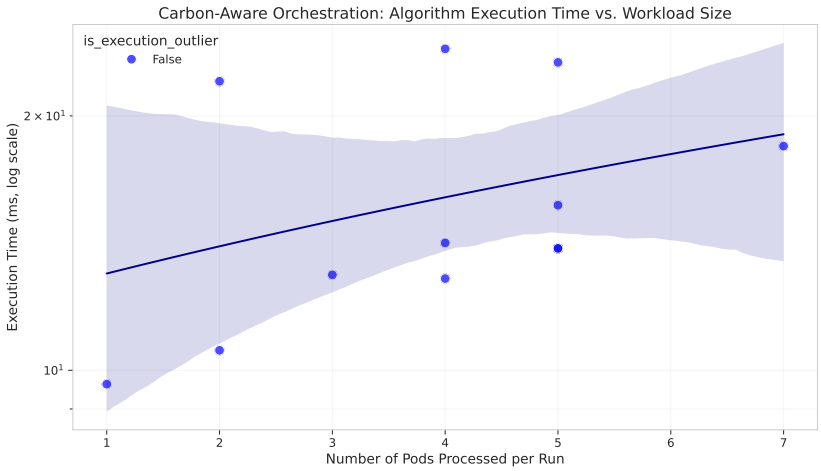

Figure 1: Execution time of the carbon-aware orchestration algorithm as a function of workload size. Note the logarithmic scale on the y-axis. While most data points show consistent performance, there's a notable outlier with significantly higher execution time that warrants further investigation.

7.2 Carbon Efficiency Analysis

The primary goal of our orchestration system is optimizing carbon efficiency. Figure 3 shows the carbon emissions per pod across different workload sizes.

We observe significant variations in per-pod emissions across different runs, ranging from approximately 0.1 to 4.2 kg CO_2e per pod. This variation can be attributed to:

- Different pod resource requirements and durations
- Varying carbon intensity in the regions where pods are placed
- Hardware heterogeneity in the target infrastructure

There appears to be an increasing trend in per-pod emissions as more pods are placed, suggesting that as infrastructure utilization increases, the algorithm must make less optimal placement decisions from a carbon perspective.

7.3 Multi-dimensional Performance Analysis

To better understand the relationships between execution time, carbon efficiency, and system utilization, Figure 4 presents these metrics across sequential algorithm runs.

Key observations from this analysis:

- CPU utilization increases steadily as more pods are added to the system

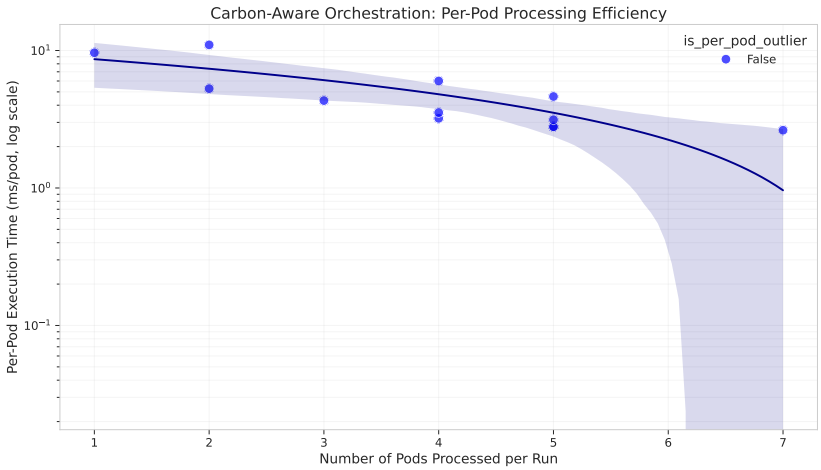

Figure 2: Per-pod execution time showing how efficiently the algorithm processes individual workload units. The outlier affects per-pod efficiency significantly.

- Per-pod carbon emissions show significant variation but generally increase as the system becomes more utilized
- The outlier in execution time (call 11) does not correspond to any significant anomaly in carbon efficiency or CPU utilization

7.4 Algorithmic Complexity Analysis

To characterize the theoretical complexity of our algorithm, Figure 5 compares our empirical results against standard complexity classes.

Excluding outliers, our algorithm exhibits behavior closest to linear complexity (O(n)) across the tested workload sizes. This indicates good scalability characteristics for practical deployment scenarios, even as the number of pods increases.

7.5 Resource Utilization and Carbon Efficiency

Figure 6 explores the relationships between resource utilization and carbon emissions.

These visualizations reveal important insights:

- Total carbon emissions generally increase with CPU utilization, as expected
- Memory utilization shows a less clear correlation with emissions than CPU utilization

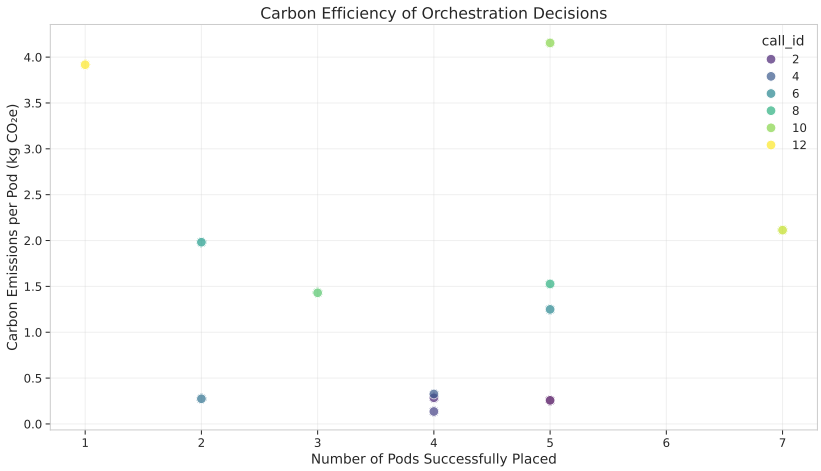

Figure 3: Carbon emissions per pod as a function of the number of pods successfully placed. This shows the variation in emissions efficiency across different workloads.

- Per-pod emissions tend to increase at higher utilization levels, indicating diminishing returns in carbon efficiency as the system approaches capacity

7.6 Discussion of Findings

Our experimental results demonstrate that the heuristic carbon-aware orchestration algorithm achieves good performance characteristics for practical workload sizes, with execution times generally remaining under 20ms. The algorithm exhibits approximately linear complexity, making it suitable for deployment in production environments.

However, the presence of outliers in execution time suggests that certain combinations of pods and constraints can lead to computational challenges. Further optimization could target these edge cases to provide more consistent performance.

From a carbon efficiency perspective, we observe that the algorithm makes effective placement decisions that consider both operational and embodied carbon emissions. However, carbon efficiency decreases as system utilization increases, highlighting the fundamental trade-off between resource efficiency and carbon optimization.

Future work will compare these results against a global optimal algorithm based on mixed-integer linear programming (MILP) to quantify how close our heuristic comes to optimal solutions across different scenarios.

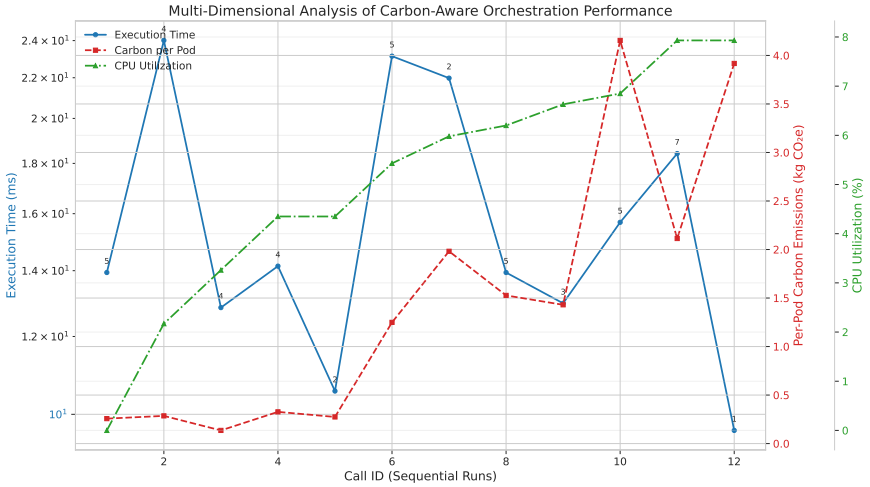

Figure 4: Multi-dimensional analysis showing execution time, carbon emissions per pod, and CPU utilization across sequential algorithm runs. Numbers above data points indicate the number of pods processed in each run.

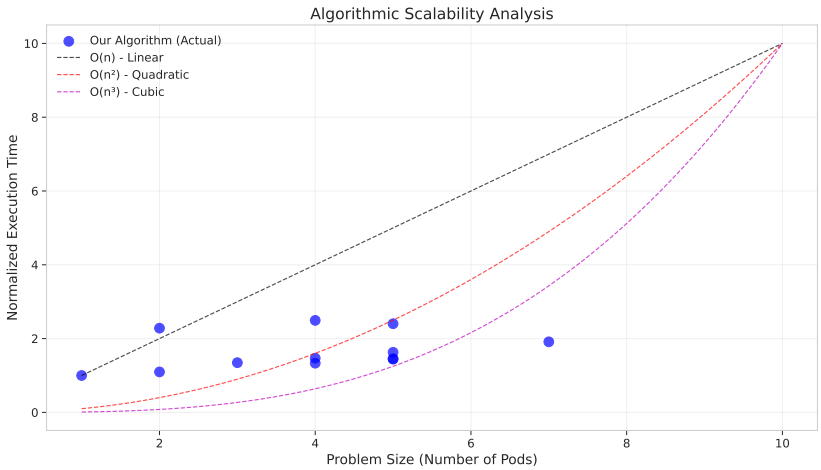

Figure 5: Comparison of the carbon-aware orchestration algorithm's performance against theoretical complexity classes. The algorithm's behavior most closely resembles linear complexity for the tested workload sizes.

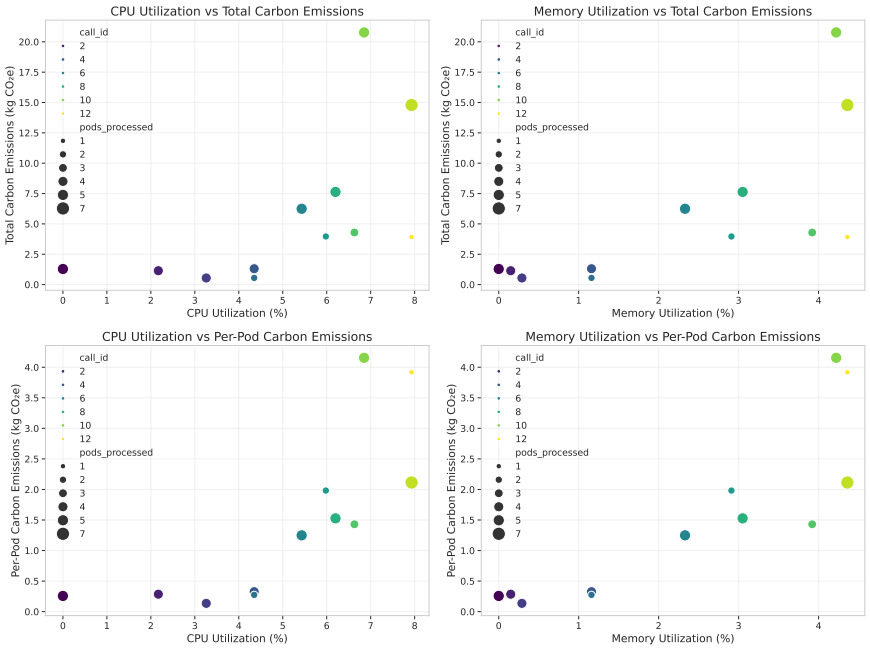

Figure 6: Relationships between resource utilization metrics (CPU and memory) and carbon emissions (both total and per-pod). Point sizes represent the number of pods processed.